

Politecnico di Milano

Anno accademico 2012-2013

Ingegneria del Software – Appello del 05 Settembre 2013

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1 (punti 16)

Si consideri un'applicazione per il prestito di libri. Ogni libro è presente in diverse copie; ogni utente iscritto al sistema può richiedere il prestito di uno dei libri presenti a catalogo, con il vincolo che al massimo può detenere tre libri complessivamente e comunque non più di una copia per ogni libro.
Si considerino le classi seguenti:

```
public class Libro {
    ...

    public int copie()
    // ritorna il numero di copie esistenti del libro, in prestito o disponibili.

    public ArrayList<Prestito> prestitiAttivi()
    // ritorna l'insieme di prestiti attivi

    public ArrayList<Utente> utentiInAttesa()
    // ritorna la lista d'attesa in ordine cronologico

    public Prestito richiestaPrestito(Utente u) throws CopieNonDisponibiliException, UtenteN
    // ritorna la richiesta di un prestito

    public void restituzionePrestito(Utente u) throws UtenteNonHaPresoInPrestitoException;
    //permette la restituzione della copia del libro prestata ad un utente u.
}

public class Prestito {
    ...

    public Libro libro()
    // ritorna il libro chiesto in prestito

    public utente()
    // ritorna l'utente che ha chiesto il prestito.
}
```

Il metodo `richiestaPrestito` é definito in questo modo: Se esiste almeno una copia disponibile, e l'utente può richiederne il prestito non avendo superato il massimo, il prestito viene generato e restituito, si decrementa il numero di copie disponibili e l'utente può iniziare la lettura; se invece l'utente non può richiederne il prestito viene generata l'eccezione `UtenteNonAbilitatoException`; Se infine non esiste una copia disponibile, viene generata l'eccezione `CopieNonDisponibiliException`. In quest'utlimo caso, l'utente viene messo in lista d'attesa (gestita con una semplice politica *first-in first-out*) e sarà poi avvisato non appena una copia dovessere essere disponibile.

Il metodo `restituzionePrestito` corrisponde alla restituzione della copia del Libro da parte di un utente; lancia l'eccezione UtenteNonHaPresoInPrestitoException qualora l'utente non risulti fra i detentori di una copia.

a) Come modifichereste le classi `Libro` e `Prestito` per notificare il primo utente in lista d'attesa non appena una copia del libro fosse restituita?

b) Si definiscano in JML le specifiche per i metodi `richiestaPrestito` e `restituzionePrestito`

c) Si supponga che esista una classe `Catalogo`, che contiene tutti i libri della bilioteca.

```
public class Catalogo {
   ...

   private ArrayList<Libro> elenco; //contiene  tutti  e soli i libri.

   public Iterator<Libro> disponibili()
   //restituisce un iteratore che itera solo sui libri disponibili, in un ordine qualsias
}
```

Si implementi l'iteratore `disponibili()`, definendo anche la classe che implementa l'iteratore e i suoi metodi.
La definizione di opportuni metodi *helper* potrebbe semplificare la definizione dell'iteratore.

Esercizio 2 (punti 6)

Notoriamente, il cibo viene mangiato. Alcuni cibi vengono mangiati solo crudi, altri solo dopo essere stati cotti, altri ancora potrebbero essere mangiati sia crudi che cotti. Volendo definire un'opportuna gerarchia di cibi, quali classi definireste?

1. È possibile definire una gerarchia di ereditarietà che rispetti il principio di sostituibilità di Liskov?

2. Se voleste definire una classe *astratta* quale gerarchia definireste? Quali metodi potreste implementare nella classe astratta?

3. E se voleste definire un'interfaccia?

Esercizio 3 (punti 4)

Si consideri il seguente frammento di codice Java e si definisca l'output prodotto, motivando brevemente la risposta.:

```
public abstract class Figura {
   public void componi(Figura f) {
      System.out.println(this + "/" + f);
   }
}

public class Quadrato extends Figura {
   public String toString() {
      return("Quadrato");
   }

   public void componi(Quadrato f) {
      System.out.println(this + "+" + f);
   }
}

public class Rettangolo extends Quadrato {
   public String toString() {
      return("Rettangolo");
   }

   public void componi(Rettangolo f) {
      System.out.println(this + "/+/" + f);
   }
}

public class Cerchio extends Figura {
   public String toString() {
      return("Cerchio");
   }
}

public class Ellisse extends Cerchio {
   public String toString() {
      return("Ellisse");
   }
}

public class Test {
   public static void main(String[] args) {
      Figura f;
      Quadrato q1, q2;
      Rettangolo r;
      Cerchio c;
      f = new Rettangolo();
      q1 = new Quadrato();
      q2 = new Rettangolo();
      r = new Rettangolo();
      c = new Ellisse();
      q1.componi(r);
      q1.componi(q2);
      q1.componi(c);
      q1.componi(f);
      r.componi(q2);
   }
}
```

Esercizio 4 (punti 6)

Si consideri il seguente frammento di codice Java:

```
public static int f(int i, int d) {
   int c = 1;

   while (c <3) {
      if (i > c && d != 0) i = c/d;
      else return i;
      c++;
   }
   return i;
}
```

1. Si definisca il diagramma del flusso di controllo.
2. Si definisca l'insieme minimo di casi di test per coprire: (a) le istruzioni, (b) le decisioni (*branch*) e (c) i cammini.